



Université du Québec
à Chicoutimi

Jalon 2

8INF970 - Atelier pratique en cybersécurité II

Fehmi Jaafar Samuel Desbiens

Justin Bossard

Mattéo Gouhier

Paul Mathé

Samuel Plet

Léo Raclet

24 février 2026

Table des matières

1	Introduction	2
2	Gestion de Projet et Planification	3
2.1	Backlog et Division du travail	3
2.2	Diagramme de Gantt et Calendrier de réalisation	4
3	Frontend	5
3.1	Maquette conceptuelle du produit	5
3.2	Stack Frontend	6
3.3	Structure	6
3.4	Page d'accueil	6
3.5	Page de résultat	7
3.6	Design	7
3.7	Transitions et chargement	7
3.8	Liaison Frontend/Backend	8
3.8.1	Configuration du port — vite.config.ts	8
3.8.2	CORS — backend/app/main.py	8
3.8.3	Limite de taille du texte — backend/app/api/models.py	8
3.8.4	Fonction runAnalysis — App.tsx	8
3.9	Limitations	9
4	Serveur et API	10
4.1	Structure	10
4.2	API	11
4.3	Prédiction	11
5	Architecture Backend API FIRST	12
5.1	Responsabilités du Module	12
5.2	Architecture Interne du Code	12
5.2.1	Le Point d'Entrée — main.py	12
5.2.2	Le Service de Communication IA — ai_service.py	12
5.3	Choix du Modèle	12
5.4	Flux de Traitement des Données	12
5.5	Format de Sortie du Modèle	13
5.6	Format de Sortie du Module	13
5.7	Sécurité & Performance	13
5.8	Perspectives d'Évolution	13
6	Conclusion	14

Partie 1

Introduction

Ce rapport s'articule autour de trois axes principaux :

1. La planification et la gestion de projet, détaillant le backlog et le calendrier de réalisation.
2. La conception frontend, exposant les maquettes conceptuelles et les choix technologiques pour l'interface utilisateur.
3. L'architecture backend, décrivant la mise en place de l'API, le flux de traitement des données et l'intégration initiale du modèle d'intelligence artificielle.

L'objectif de cette étape est de démontrer la mise en place d'un prototype fonctionnel (MVP) capable d'effectuer une vérification de base.

Pour rappel, notre projet est hébergé publiquement sur GitHub : <https://github.com/realnitsuj/fake-news-detection>.

Pour tester, il suffit de cloner le répertoire, puis :

- Pour le frontend, aller dans le dossier `frontend/`, et exécuter :

```
npm install && npm run dev
```

Le frontend sera disponible sur <http://localhost:5173>.

- Pour le backend, aller dans `backend/`, et exécuter :

```
uv python install 3.12 && uv run fastapi dev
```

Le backend sera disponible sur <http://127.0.0.1:8000>.

Partie 2

Gestion de Projet et Planification

Pour assurer le bon déroulement du développement de **VerifAI** et respecter les échéances du plan de cours, nous avons opté pour une approche de gestion hybride. Nous utilisons l'outil **GitHub Projects**, qui nous permet de maintenir un backlog détaillé (méthodologie Agile) tout en générant automatiquement un diagramme de Gantt pour la planification macroscopique.

2.1 Backlog et Division du travail

Le projet a été découpé en tâches techniques précises. Pour répondre aux exigences de réalisation en équipe, chaque tâche de notre backlog est assignée à un responsable clair, garantissant une division équitable du travail.

Le tableau de bord utilise un système d'étiquettes (Labels) pour catégoriser les pôles d'expertise :

- **Développement Backend & Architecture (backend, archi, dev)** : Géré principalement par Mattéo GOUHIER (ex : mise en place de l'API FastAPI, codage du prototype MVP).
- **Interface & Expérience Utilisateur (frontend, design)** : Assuré par Paul MATHÉ (intégration de l'interface graphique) et Justin BOSSARD (création des maquettes).
- **Gestion & Recherche (gestion, recherche)** : Suivi administratif et bibliographique piloté par le reste de l'équipe pour assurer la conformité avec les jalons.
- **Données & Intelligence Artificielle (data, IA, science)** : Phase de sélection des datasets et protocoles préparatoires pour le modèle de détection.

Title	Labels	Assignees	Status	Linked pull requests	Sub-issues progress	Start	End	Milestone
1 Créer le nom de la startup et le logo #1	gestion	PumpeDie	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
2 Compléter l'état de l'art sur la détection de Fake News (Recherche biblio) #2	recherche	Go-GoZ...	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
3 Sélectionner les datasets (ex: LIAR, FakeNewsNet) #3	data	PumpeDie	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
4 Choisir la stack technique (Python/Flask?, React?, PyTorch/tensorflow?) #4	archi	leoraclet	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
5 Expérimentations possibles (supervisé vs outliars, persuasion IA) #21	test	paulm123...	Done			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
6 Définir le protocole d'expérimentation (métriques de succès) #5	science		Todo			Jan 20, 2026	Jan 27, 2026	Jalon 1 : État de l'a
7 Créer les maquettes conceptuelles (Figma/Adobe XD) #6	design	realistitaj	Todo			Jan 27, 2026	Feb 3, 2026	Jalon 2 : Prototype
8 Mettre en place l'API de base (squelette) #7	backend	leoraclet	Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
9 Intégration de l'interface graphique #8	frontend	paulm123...	Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
10 Concevoir le GANTT détaillé #9	gestion	PumpeDie	Done			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
11 Codifier le prototype initial (MVP) #10	dev	leoraclet...	In Progress		0 / 3	Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
12 Connexion Front-Back #20			Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
13 Création des routes API (Login, Post News) #19			Todo			Feb 3, 2026	Feb 24, 2026	Jalon 2 : Prototype
14 Création de la base de données #18			Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
15 Implémentation du modèle de détection (Données temporelles) #11	IA		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
16 Intégration du LLM pour l'explication automatisée des résultats #12	IA		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
17 Rédiger l'étude de marché et identifier les concurrents #13	marketing		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
18 Nettoyage et préparation finale des données d'entraînement #14	data		Todo			Feb 24, 2026	Mar 31, 2026	Jalon 3 : IA & Mar
19 Évaluation de la performance vs concurrents #15	test		Todo			Mar 31, 2026	Apr 28, 2026	Rendu Final : Procl
20 Tests de facilité d'utilisation #16	test		Todo			Mar 31, 2026	Apr 28, 2026	Rendu Final : Procl
21 Rédaction du rapport final complet #17	documentation		Todo			Mar 31, 2026	Apr 28, 2026	Rendu Final : Procl

FIGURE 2.1 : Vue Backlog - Division du travail et état d'avancement

Chaque tâche est associée à un Jalon cible (Milestone) et possède un statut de progression (*Todo, In Progress, Done*).

2.2 Diagramme de Gantt et Calendrier de réalisation

La vue chronologique de GitHub Projects nous permet de visualiser les dépendances et de paralléliser nos efforts. Le calendrier est structuré autour des quatre grandes phases du projet :

Phase 1 : Idéation et État de l'art (Jalon 1 - 20 Jan. au 27 Jan.) - *[Terminé]*

- Création de l'identité de la startup (Nom et Logo VerifAI).
- Recherche bibliographique et sélection du dataset (WELFake).
- Choix de la stack technique (Python, PyTorch, Vite).

Phase 2 : Prototypage et Maquettage (Jalon 2 - 27 Jan. au 24 Fév.) - *[En cours]* Cette phase concentre l'essentiel de nos efforts actuels de développement.

- **27 Jan - 03 Fév** : Création des maquettes conceptuelles de l'interface.
- **03 Fév - 24 Fév** : Développement en parallèle du Frontend (interface graphique) et du Backend (mise en place de l'API de base et création des routes de test).
- **Mi-Février** : Connexion Front-Back et validation du Prototype Minimum Viable (MVP).

Phase 3 : Intelligence Artificielle (Jalon 3 - 24 Fév. au 31 Mars) - *[À venir]* Une fois le prototype fonctionnel validé, l'effort basculera sur le moteur scientifique.

- Création de la base de données finalisée.
- Implémentation et entraînement du modèle de détection de Fake News.
- Intégration du LLM (Large Language Model) pour la génération automatique d'explications destinées aux utilisateurs.
- Rédaction de l'étude de marché marketing.

Phase 4 : Tests et Rendu Final (31 Mars au 28 Avril) - *[À venir]*

- Évaluation des performances du modèle face aux concurrents.
- Tests de facilité d'utilisation de la plateforme.
- Rédaction du rapport final complet et préparation de la soutenance.

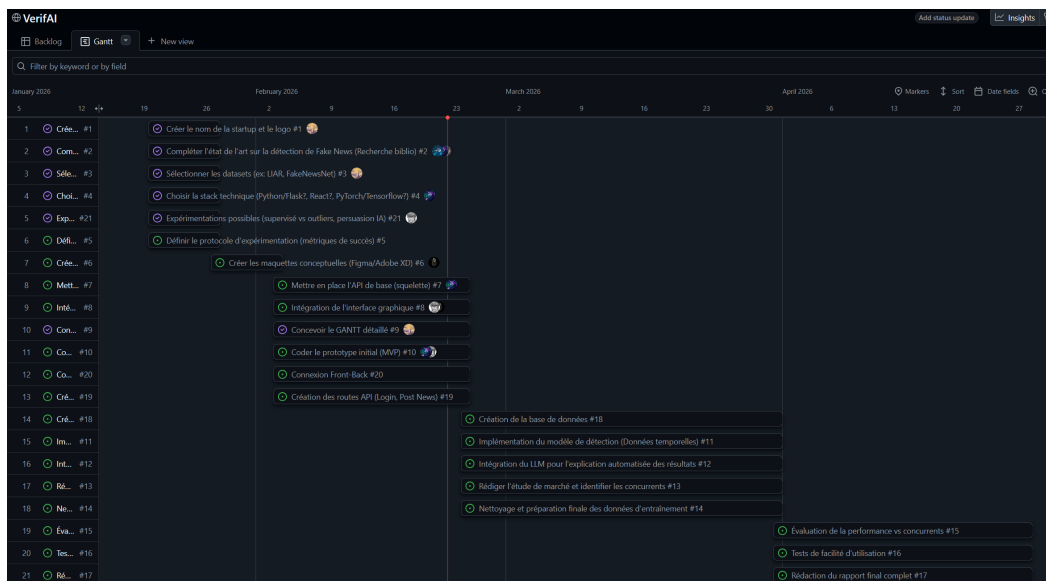


FIGURE 2.2 : Vue Gantt - Planification temporelle des Jalons

Partie 3

Frontend

3.1 Maquette conceptuelle du produit

Pour notre produit, nous avons cherché à avoir une interface simple et épurée, qui va à l'essentiel. Voici la maquette conceptuelle de notre interface, en deux pages principales, accueil et résultats :

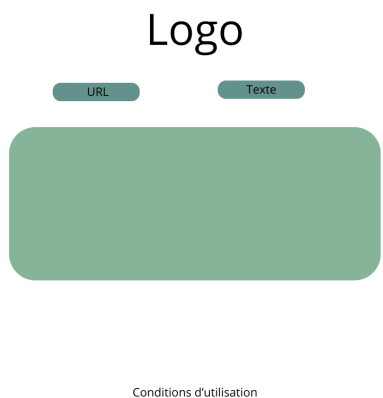


FIGURE 3.1 : Maquette conceptuelle de l'accueil de VerifAI

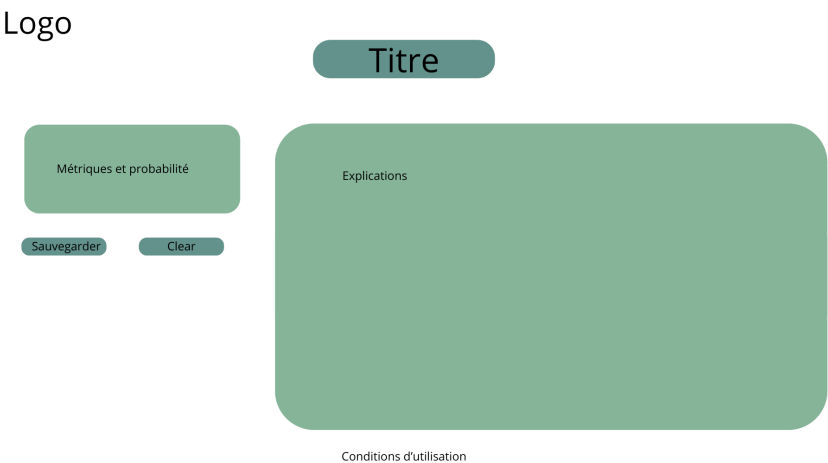


FIGURE 3.2 : Maquette conceptuelle des résultats

3.2 Stack Frontend

Le frontend a été développé avec **React 19** et **TypeScript**, bundlé via **Vite 7** configuré sur le port 8080. L'interface utilise **Tailwind CSS v4** pour le style et **shadcn/ui** (style New York) pour les composants de base comme les boutons, les inputs et les textareas. Les icônes proviennent de **Lucide React**.

3.3 Structure

Par souci de simplicité, l'ensemble du code frontend est centralisé dans un seul fichier `src/App.tsx`, sans pages ou composants séparés. Cela facilite la lecture et la maintenance pour un projet de cette taille.

```
src/
|- App.tsx          <- global : types, logique, composants, pages
|- index.css        <- thème + animations
|- assets/
    |- logo.jpg
```

Le fichier `App.tsx` est organisé dans l'ordre suivant : 1. Types TypeScript (`Verdict`, `AnalysisResult`) 2. Fonction `runAnalysis` — appel à l'API backend 3. Configuration des verdicts (FAKE / REAL / UNCERTAIN) avec leurs couleurs et labels 4. Composant `BgDecorations` — décoration de fond partagée entre les deux pages 5. Composant `MetricRow` — barre de progression réutilisable 6. Fonction `HomePage` — page d'accueil 7. Fonction `ResultPage` — page de résultat 8. Export `App` — gestion du routing et de l'état global

3.4 Page d'accueil

La page d'accueil est épurée et centrée. Elle affiche le logo VerifAI avec un effet de glow bleu, un sous-titre, puis une carte principale contenant les éléments de saisie. L'utilisateur peut basculer entre deux modes via des tabs : **URL** pour coller un lien et **Texte** pour coller directement le contenu d'un article. En dessous de la carte, trois statistiques (précision, délai, nombre de sources) donnent de la crédibilité à l'outil. Un lien vers les conditions d'utilisation est placé en pied de page.

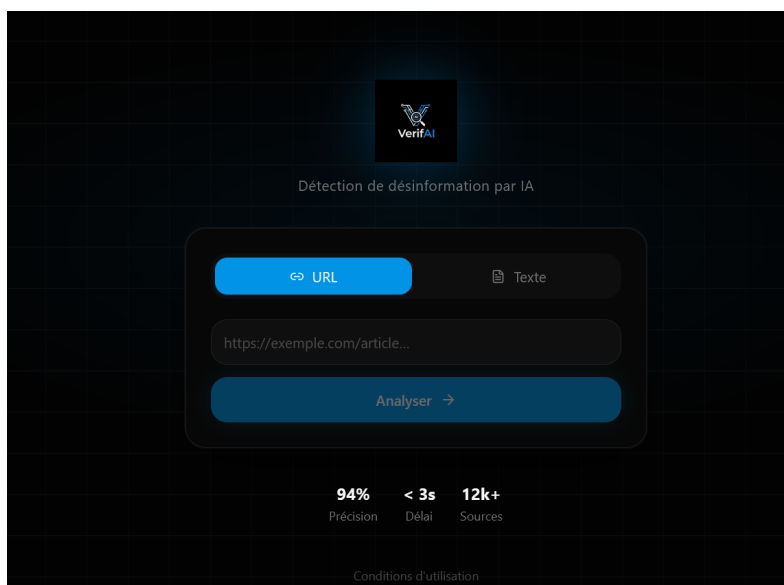


FIGURE 3.3 : Capture d'écran du prototype - page d'accueil

3.5 Page de résultat

La page de résultat s'organise en trois zones distinctes. En haut, une **carte verdict pleine largeur** affiche le résultat principal avec une couleur adaptée au verdict (rouge pour FAKE, vert pour REAL, ambre pour UNCERTAIN), le score de confiance en grand et une barre de progression animée.

En dessous, un **layout en deux colonnes** permet de lire les informations en parallèle. La colonne gauche (plus étroite) contient un cercle de score et quatre barres de métriques détaillées. La colonne droite (plus large) affiche l'explication textuelle du résultat ainsi que les sources consultées par le modèle. En bas de page, deux boutons permettent de **sauvegarder** le rapport au format `.txt` ou de **revenir** à l'accueil.

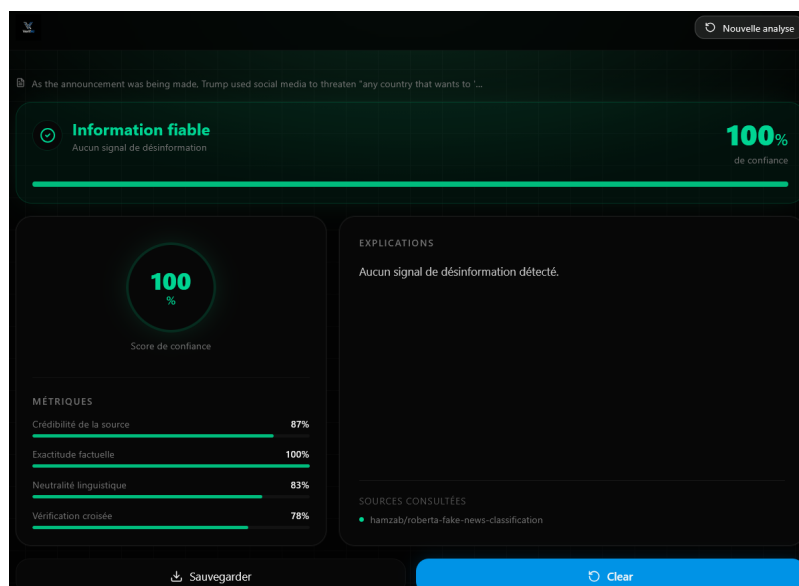


FIGURE 3.4 : Capture d'écran du prototype - page de réponse

3.6 Design

Le design suit l'identité visuelle du logo VerifAI — fond très sombre (`oklch(0.09 0 0)`) et bleu électrique comme couleur primaire (`oklch(0.62 0.19 235)`). Chaque page possède un glow radial en fond dont la couleur change selon le verdict affiché, renforçant visuellement le résultat. Une grille subtile en arrière-plan ajoute de la profondeur sans surcharger l'interface. La police **Inter** est chargée depuis Google Fonts pour une typographie propre et lisible.

3.7 Transitions et chargement

Un soin particulier a été apporté à la fluidité des transitions entre les pages. Au lieu d'attendre la réponse du backend avant de changer de vue — ce qui provoquerait un écran noir ou un blocage visible — le frontend **bascule immédiatement** vers la page de résultat dès que l'utilisateur clique sur Analyser, et affiche un skeleton en attendant la réponse.

```
const handleAnalyze = async (input: string) => {
  setResult(null)          // efface le résultat précédent
  setShowResult(true)     // navigue instantanément vers la page résultat
  const data = await runAnalysis(input)
  setResult(data)          // remplit les données quand le backend répond
}
```

Le skeleton comprend un **spinner SVG circulaire animé** à la place du cercle de score, des **barres**

grises pulsantes pour les métriques et des **lignes grises de largeurs variées** pour simuler le texte des explications. Dès que la réponse arrive, les données remplacent le skeleton sans transition brusque.

3.8 Liaison Frontend/Backend

3.8.1 Configuration du port — vite.config.ts

Vite est configuré pour démarrer sur le port 8080, qui est déjà autorisé dans les origines CORS du backend — aucune modification côté backend n'est donc nécessaire pour le CORS.

```
server: {  
  port: 8080,  
}
```

3.8.2 CORS — backend/app/main.py

Le port 8080 étant déjà présent dans la liste des origines autorisées, aucune modification n'a été nécessaire de ce côté.

```
origins = [  
    "http://localhost",  
    "http://localhost:8080", # déjà présent  
]
```

3.8.3 Limite de taille du texte — backend/app/api/models.py

La contrainte `max_length` devra être augmentée de 500 à 5000 caractères pour permettre l'analyse d'articles complets, car les textes réels dépassent facilement 500 caractères.

```
# Avant  
text: str = Field(..., min_length=50, max_length=500)  
  
# Après  
text: str = Field(..., min_length=50, max_length=5000)
```

3.8.4 Fonction `runAnalysis` — App.tsx

La fonction `runAnalysis` est le point de jonction entre le frontend et le backend. Elle envoie le texte en POST sur `/ai/check-text`, récupère le verdict et le score, puis construit l'objet `AnalysisResult` attendu par la page de résultat. Le score retourné par le backend est une chaîne du type `"87.43%"` — le `replace("%", "")` est nécessaire avant le `parseFloat` pour éviter un NaN.

```
async function runAnalysis(input: string): Promise<AnalysisResult> {  
  const res = await fetch("http://localhost:8000/ai/check-text", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ text: input }),  
  })  
  if (!res.ok) throw new Error(`Erreur ${res.status}`)  
  const { result } = await res.json()  
  const confidence = Math.round(parseFloat(result.score.replace("%", "")))  
  const verdict: Verdict = result.prediction === "FAKE" ? "FAKE" : "REAL"  
  return {  
    verdict,  
    confidence,  
  }  
}
```

```

inputPreview: input.length > 100 ? input.slice(0, 100) + "..." : input,
isUrl: input.startsWith("http"),
metrics: {
  sourceCredibility: verdict === "FAKE" ? 21 : 87,
  factualAccuracy: confidence,
  linguisticBias: verdict === "FAKE" ? 14 : 83,
  crossVerification: verdict === "FAKE" ? 11 : 78,
},
explanation: verdict === "FAKE"
  ? "Le modèle a détecté des signaux de désinformation."
  : "Aucun signal de désinformation détecté.",
sources: ["hamzab/roberta-fake-news-classification"],
}
}

```

3.9 Limitations

Plusieurs limitations ont été identifiées au cours du développement.

Le **mode URL** ne fonctionne pas comme attendu : le frontend envoie l'URL comme une chaîne de texte brute, sans en extraire le contenu. Le modèle reçoit donc l'URL elle-même à analyser, ce qui produit des résultats incohérents. Pour corriger cela, il faudrait ajouter un mécanisme de scraping côté backend (par exemple avec `beautifulsoup4`) pour récupérer le contenu de la page avant de l'envoyer au modèle.

Les **métriques détaillées** sont en partie statiques. Le backend ne retournant qu'un score global et un label, les valeurs de crédibilité de source, neutralité linguistique et vérification croisée sont calculées à partir de valeurs fixes selon le verdict. Seule l'exactitude factuelle reflète le vrai score du modèle.

La contrainte **min_length=50** côté backend peut surprendre l'utilisateur si son texte est trop court — une validation côté frontend avant l'envoi éviterait une erreur 422 silencieuse.

Enfin, le modèle HuggingFace peut mettre **10 à 30 secondes** à répondre lors de la première requête après une période d'inactivité (cold start), ce qui peut donner l'impression que l'application est bloquée.

Partie 4

Serveur et API

Le backend met en œuvre une **API personnalisée** permettant notamment au frontend d'effectuer des actions telles que la vérification de texte ou le téléversement de fichiers.

4.1 Structure

L'image suivante présente l'architecture actuelle de notre solution. Elle montre comment s'articule le rôle de notre serveur au sein du système, ainsi que la manière dont s'organisent les interactions entre l'IA et l'utilisateur à travers celui-ci.

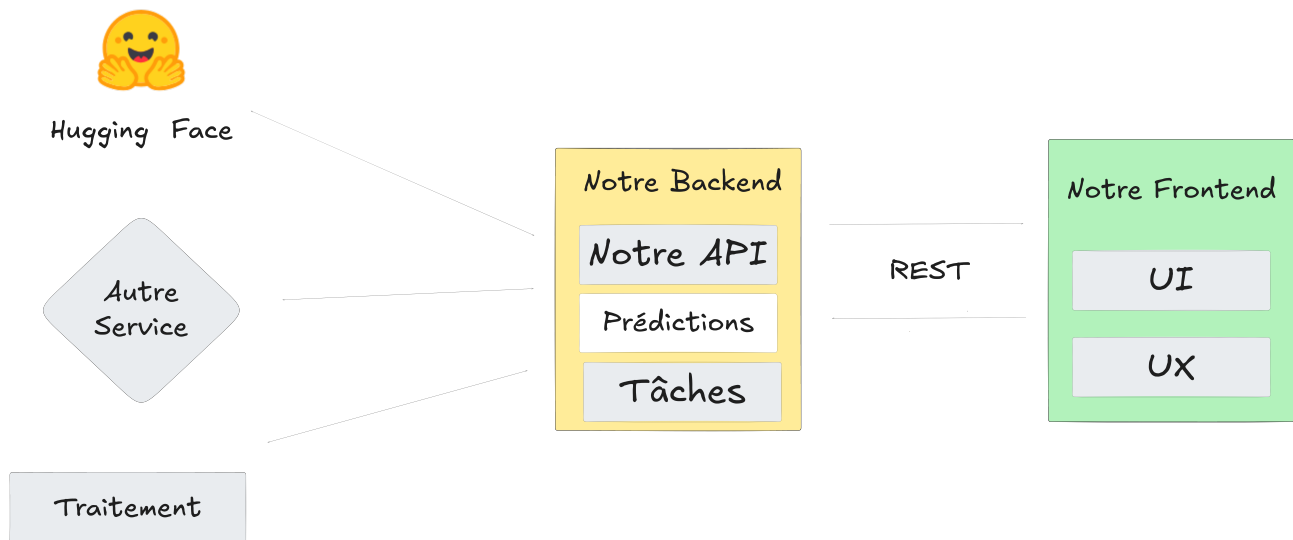


FIGURE 4.1 : Schema Technique du Projet

Le serveur, qui implémente une REST API, agit comme intermédiaire entre le Frontend, le traitement des données utilisateurs et les tâches de prédiction de l'IA sur ces données.

Son rôle est de centraliser les secrets, tels que les clés API, requis pour les requêtes de prédiction. Il permettrait également d'effectuer des traitements avancés sur les diverses sources de données de l'utilisateur, comme l'extraction de texte à partir de fichiers PDF ou d'images grâce à des scripts et à l'OCR.

Pour l'interaction avec le frontend, le serveur expose une API REST à laquelle le frontend se connecte pour exécuter des actions et récupérer ou envoyer des données.

4.2 API

L'API est développée avec **FastAPI**, choisi pour :

- Son moteur asynchrone, idéal pour l'intégration d'IA et l'inférence.
- Sa compatibilité native avec **Pydantic**, qui facilite le typage des données.

Elle se structure autour de **deux catégories d'endpoints** :

- **/ai** : pour les prédictions et traitements liés à l'IA sur les données utilisateur.
- **/files** : pour la gestion de fichiers (images, PDF, etc.), notamment dans le cadre de la vérification de données provenant de sources variées.

4.3 Prédiction

Parmi les tâches assurées par le serveur figure l'inférence du modèle d'IA sur les données utilisateur. Ce processus repose sur un script Python qui effectue une requête vers l'API de Hugging Face, analyse la réponse obtenue, puis transmet les résultats au serveur, lequel les renvoie finalement à l'utilisateur.

Partie 5

Architecture Backend API FIRST

5.1 Responsabilités du Module

Le backend remplit trois fonctions principales :

1. **Sécurisation des secrets** : Gestion du Token API via variables d'environnement (`.env`).
2. **Normalisation** : Transformation du texte brut en format compatible pour l'IA.
3. **Traduction des résultats** : Transformer le retour via l'API en texte compréhensible par l'utilisateur.

5.2 Architecture Interne du Code

Le backend est segmenté en **deux composants** pour garantir la maintenabilité et éviter le code spaghetti.

5.2.1 Le Point d'Entrée — `main.py`

C'est lui qui contrôle l'application. Il reçoit les données entrantes, y applique les règles de validation (longueur minimale du texte), et construit la réponse finale suivant un schéma de réponse prédéfini.

5.2.2 Le Service de Communication IA — `ai_service.py`

Ce service est le point central de la connexion à l'IA :

- **Gestion des requêtes HTTP** : Utilisation de `requests` pour dialoguer avec le Cloud.
- **Routage Dynamique** : Connexion au Router Hugging Face pour optimiser la disponibilité.
- **Gestion de la résilience** : Paramétrage du `wait_for_model` pour éviter les plantages lors du chargement des modèles IA sur le serveur distant.

5.3 Choix du Modèle

Nous avons choisi un modèle réputé basé sur l'architecture **RoBERTa**, enrichi sur des articles comportant ou non des fake news. Ce choix n'est pas définitif, mais permet d'avoir une vision claire du fonctionnement de notre code.

Modèle utilisé : [hamzab/roberta-fake-news-classification](#)

5.4 Flux de Traitement des Données

Le backend transforme le texte brut en verdict compréhensible en **cinq étapes** :

Texte brut → Validation → Authentification → Analyse IA → Parsing → Post-traitement → JSON

1. Lecture du texte : Le système réceptionne le texte et valide son format (longueur minimale) pour filtrer les requêtes inutiles.
2. Authentification : Il injecte de manière sécurisée la clé API pour autoriser la communication avec les serveurs de Hugging Face.
3. Traitement par L'IA : Le texte est transmis au modèle RoBERTa, qui réalise une analyse sémantique profonde pour détecter les marqueurs de désinformation.
4. Parsing : Le backend simplifie la réponse de l'IA (listes imbriquées) pour n'en extraire que les valeurs essentielles : le label et le score.
5. Post-traitement : Il traduit ces statistiques en information humaine. Le score est converti en pourcentage et le label technique devient un verdict clair et lisible. Ce flux garantit que l'interface finale reçoit une donnée **fiable, lisible et prête à être affichée**.

5.5 Format de Sortie du Modèle

Le backend ne renvoie pas de texte brut, mais un **objet JSON standardisé**. Cette structure permet de connecter n'importe quel Frontend sans modifier le code serveur.

```
{
  "is_fake": true,
  "confidence_score": 0.9997,
  "label_detected": "FAKE",
  "message": "Attention, ce contenu semble suspect.",
  "model_version": "hamzab/roberta-fake-news-classification"
}
```

5.6 Format de Sortie du Module

En sortie, on renvoie pour le moment dans la CLI le texte à analyser que l'on redonne à l'utilisateur pour qu'il puisse relire si besoin, le résultat sous forme de phrase (**FAKE NEWS détectée !** ou **Information fiable**), le score de confiance et le label, donc Fake ou Vrai (cela peut sembler redondant mais permet de vérifier que lors de la conversion depuis le json aucune erreur ne se produit).

5.7 Sécurité & Performance

Pour garantir la sécurité nous stockons les informations importante dans des variables d'environnement (dans `.env`) pour empêcher la fuite des clés API sur les dépôts de code publics. Et pour permettre d'améliorer la performance, nous avons construit l'application afin de pouvoir facilement changer le modèle (ici une instance de Roberta), à la seule condition de changer le format que l'on récupère sans le JSON.

5.8 Perspectives d'Évolution

Nous allons par la suite, essayer de changer le modèle pour prendre un modèle de type LLM, permettant de justifier le verdict. Cela permettrait d'afficher ces détails aux utilisateurs pour mieux comprendre les raisons derrière un résultat. On pourra le faire assez facilement, grâce à l'architecture expliqué ci-dessus. Et sur les bases de ce modèle, nous allons essayer de faire du fine-tuning avec d'autres bases de données.

Partie 6

Conclusion

Ce jalon a permis de valider les fondations techniques de la solution VerifAI. L'architecture backend, basée sur FastAPI et une approche modulaire, est opérationnelle et prête à traiter les requêtes d'inférence. La gestion de projet est stabilisée grâce à une méthodologie hybride et un suivi rigoureux sur GitHub.

Le jalon 3 se concentrera sur l'optimisation du modèle d'intelligence artificielle et l'ajout de fonctionnalités explicatives (LLM). Le prototype actuel constitue une base solide pour le développement du produit commercial final.